

# IBM Data Engineering Course | Coursera

## Machine Learning With Apache Spark

### Summary

---

#### Module One

#### Get started With Machine Learning

#### Get started With Machine Learning

#### Lesson 1: Introduction to machine learning for every one

#### Objectives

- Define *machine learning*.
  - Explain how *machine learning* works.
  - Describe real-world *use cases*.
  - Differentiate between *AI*, *machine learning*, and *deep learning*.
  - List the *categories* and *branches* of machine learning.
- 

#### Key Concepts / Steps

- **Definition:**  
Machine learning (ML) is a subfield of computer science that enables computers to learn *without being explicitly programmed*.  
Coined by Arthur Samuel at IBM in 1959.
- **How ML works (vs. traditional programming):**
  - Traditional approach: manually define rules to classify data (e.g., cats vs. dogs) → fails to generalize.
  - ML approach: extract features from data, use labeled examples to train models that learn patterns and make predictions.
- **Example – Cancer Detection:**
  - Cell samples have numerical characteristics (e.g., clump thickness, cell size).
  - A model can learn from past labeled samples (benign/malignant) to predict future cases.
  - This can aid or even outperform human diagnosis in speed and consistency.
- **Real-Life Applications:**
  - **Recommendation systems:** Netflix, Amazon.
  - **Finance:** loan approval, fraud detection.
  - **Telecom:** customer churn prediction, segmentation.

- **Others:** chatbots, face recognition in games, automated summarization.
  - **AI vs ML vs Deep Learning:**
    - **AI:** Broad field aiming to mimic human cognition (e.g., vision, language).
    - **ML:** Subfield of AI focused on statistical learning from data.
    - **Deep Learning (DL):** Subset of ML using neural networks to make independent decisions, closer to how the brain works.
  - **Main ML Categories:**
    1. **Supervised Learning** (uses labeled data):
      - *Regression:* predict continuous values (e.g., house prices).
      - *Classification:* predict categories (e.g., tumor: benign or malignant).
    2. **Unsupervised Learning** (uses unlabeled data):
      - *Clustering:* find groups or patterns (e.g., customer segments).
  - **Branches of Machine Learning:**
    - **Deep Learning:** Brain-inspired models (e.g., neural networks).
    - **Natural Language Processing (NLP):** Understanding human language.
    - **Computer Vision:** Understanding digital images.
    - **Reinforcement Learning:** Learning through rewards and penalties.
- 

## Examples

- **Cat vs. Dog Classifier:** Using features like eyes, tail, legs, etc., to classify images.
  - **Cancer Diagnosis:** Predicting if a cell is benign or malignant based on numerical characteristics.
  - **Netflix & Amazon:** Recommending content based on user behavior.
  - **Loan Approval:** Predicting likelihood of repayment using applicant data.
- 

## Key Takeaways

- ML enables computers to learn from data instead of being hand-coded.
  - It powers many modern tools — from medical diagnostics to content recommendations.
  - AI is the umbrella term → ML is a branch of AI → Deep Learning is a further subset of ML.
  - Two major learning styles: **supervised** and **unsupervised**.
  - ML has diverse branches like NLP, CV, deep learning, and reinforcement learning — each with unique use cases.
-

## Lesson 2: Role of Data Engineering in Machine Learning

### Objectives

- Define the role of **data engineering** in machine learning (ML) projects.
  - List the core **responsibilities** of data engineers across the ML pipeline.
- 

### Key Concepts / Steps

#### What is Data Engineering in ML?

- It's the foundation of machine learning, responsible for designing, building, and maintaining the **infrastructure** and **data pipelines**.
  - Ensures that data is **available**, **reliable**, and **ready** for analysis and modeling.
- 

#### Data Collection

1. **Define Requirements:** Structured/unstructured data, internal/external sources.
  2. **Identify Sources:** Databases, APIs, web scraping, manual entry.
  3. **Evaluate Data:** For quality, cost, reliability, and relevance.
  4. **Collect Data:** Using queries, scrapers, or manual methods.
  5. **Ensure Consistency:** Standardize format and structure.
- 

#### Data Preparation

1. **Exploration:** Detect anomalies, missing values, and outliers.
  2. **Cleaning:** Remove duplicates, fix typos, impute missing values.
  3. **Transformation:** Normalize, scale, and format for analysis.
- 

#### Data Integration & Formatting

1. **Integration:** Combine datasets from various sources using merging/joining.
  2. **Formatting:** Convert data into analysis-ready formats (e.g., dates → numbers).
-

## Data Storage & Management

- **Store:** Choose the right system (DBs, data lakes, cloud) based on scale and speed.
  - **Organize:** Design schemas, partitions, and indexing strategies.
  - **Secure:** Apply encryption, access control, and backup protocols.
  - **Retrieve:** Enable efficient querying with caching and indexing.
  - **Backup & Recover:** Implement regular backups and recovery mechanisms.
- 

## Feature Engineering

1. **Feature Selection:** Choose the most informative variables.
  2. **Scaling:** Standardize feature ranges to prevent bias.
  3. **Feature Construction:** Create new features (e.g., interactions, bins, polynomials).
  4. **Dimensionality Reduction:** Reduce variables using PCA or similar.
  5. **Encode Categorical Data:** One-hot encoding or label encoding.
  6. **Handle Missing Values:** Use imputation techniques (mean, interpolation).
- 

## Examples

- Collecting health records from multiple hospital systems.
  - Cleaning messy customer datasets for a churn prediction model.
  - Storing training data in a cloud-based data lake with schema validation.
  - Engineering new features like interaction terms to improve ML accuracy.
- 

## Key Takeaways

- Data engineering is the *backbone* of any ML project — without it, models can't be trained reliably.
  - Data engineers handle **collection, cleaning, transformation, storage, and retrieval** of data.
  - They ensure the data is **accurate, consistent, and scalable** for ML pipelines.
  - Feature engineering — a key task — improves model performance and interpretability.
  - Proper data infrastructure supports everything from model training to real-time predictions.
-

## Lesson 3: Machine Learning Model Lifecycle

### Objectives

- Understand the **end-to-end lifecycle** of a machine learning (ML) model.
  - Identify the key **stages** in the ML workflow from idea to deployment.
  - Recognize the **iterative nature** of ML development.
- 

### Key Concepts / Steps

#### The 5 Key Stages in the ML Lifecycle:

1. **Problem Definition**
    - Clearly define the problem you're solving.
    - Example: “Can we recommend beauty products to users based on their behavior?”
  2. **Data Collection**
    - Gather data from various sources (databases, APIs, logs, etc.).
    - This begins the *ETL* process (Extract, Transform, Load).
  3. **Data Preparation**
    - Clean, format, and transform the collected data.
    - Store it in a centralized place accessible to ML engineers.
  4. **Model Development & Evaluation**
    - Build and train machine learning models using prepared data.
    - Evaluate the model’s performance using metrics (e.g., accuracy, precision).
  5. **Model Deployment**
    - Deploy the model into production where it can serve predictions in real-time or batch processes.
- 

### Iterative Nature of the Lifecycle

- The lifecycle is *not strictly linear* — it loops:
    - If a deployed model fails or underperforms → go back to earlier steps.
    - Common iterations: revisit data collection, redefine the problem, or retrain the model.
- 

### Examples

- A beauty product recommendation system may need to revisit customer feedback → redefine the recommendation logic → recollect behavioral data → retrain and redeploy the model.

## 🌱 Key Takeaways

- The **Machine Learning Model Lifecycle** consists of five key phases:  
● Problem Definition → 📁 Data Collection → 🛠 Data Preparation → 🧠 Model Development & Evaluation → 🚀 Model Deployment
- This process is **cyclical**: feedback from production can trigger earlier stages.
- ETL (Extract, Transform, Load) is a **critical early step** to prepare high-quality data.
- A solid understanding of the full lifecycle is essential for any successful ML project.

---

## 📖 Lesson 4: Supervised vs Unsupervised Learning

### 🧠 Objectives

- Define **Supervised** and **Unsupervised Learning**.
- List common **use cases** for both types.
- Understand the concept of **Clustering**.
- Differentiate clearly between **labeled** and **unlabeled** learning approaches.

### ⚡ Key Concepts / Steps

#### 🤖 Supervised Learning

- **Definition**: Learning from *labeled data* — where both inputs (features) and outputs (labels) are known.
- **Goal**: Teach the model to predict outcomes for *new data* based on historical patterns.

#### 📁 Types of Supervised Learning:

1. **Classification**
  - Predicts *categories or class labels* (e.g., benign vs. malignant).
  - Example: "Will I pass or fail the exam?"
2. **Regression**
  - Predicts *continuous values* (e.g., CO2 emissions of a car).
  - Uses dependent and independent variables to model relationships.

## 🌱 Key Components:

- **Features**: Input variables (e.g., engine size, fuel consumption).
  - **Observations**: Rows in a dataset (a single data point).
  - **Label**: Known outcome used for training.
-

## Unsupervised Learning

- **Definition:** Learning from *unlabeled data* — no predefined outputs.
- **Goal:** Discover hidden patterns, structures, or relationships in the data.

### Techniques:

- **Clustering:**
  - Group similar data points based on shared features.
  - Used for customer segmentation, music recommendation, anomaly detection.
- **Dimensionality Reduction:**
  - Reduces redundant features for simpler analysis.
  - Makes data more manageable for modeling.
- **Market Basket Analysis:**
  - Finds associations (e.g., "people who buy X also buy Y").
- **Density Estimation:**
  - Identifies the distribution or structure in the dataset.

### Examples

- **Supervised:**
  - Medical diagnosis (benign vs malignant)
  - Predicting test scores
  - House price prediction
- **Unsupervised:**
  - Clustering customers by behavior
  - Organizing music by listening patterns
  - Detecting fraud or outliers without prior labels

---

## Key Takeaways

- **Supervised Learning**
    - ✓ Uses *labeled* data
    - ✓ Algorithms: *Classification & Regression*
    - ✓ Predictive and controlled
  - **Unsupervised Learning**
    - ✓ Uses *unlabeled* data
    - ✓ Algorithms: *Clustering, Dimensionality Reduction*
    - ✓ Exploratory and pattern-finding
  - Clustering groups similar data points based on their characteristics.
  - Supervised models are easier to validate; unsupervised models have fewer evaluation methods and offer less control.
-

## Lesson 5: Regression

### Objectives

- Define the key concepts of **regression**.
  - Identify common **regression algorithms**.
  - Interpret **regression results** (line equation, slope, intercept, etc.).
  - Differentiate between **classification vs. regression**.
  - Choose whether regression is the right approach for a given problem.
- 

### Key Concepts / Steps

#### What is Regression?

- **Regression** analyzes the **relationship between a dependent variable (target) and one or more independent variables (features)**.
- The **dependent variable is continuous**, meaning it can take on any numerical value (e.g., prices, scores, temperatures).

#### Example: Predicting Final Exam Scores

- Plot midterm vs. final grades.
- Fit a line through the points → this is your **line of best fit**.
- Equation:

$$y = mx + b$$

- **m** = slope (rise over run)
- **b** = y-intercept (value of y when x = 0)

### Types of Regression

1. **Simple Linear Regression**
  - One feature (e.g., midterm score) → one outcome (final grade).
2. **Multiple Linear Regression**
  - Multiple features (e.g., midterm + attendance) → one outcome.
  - Graph becomes multi-dimensional (e.g., 3D or more).
3. **Polynomial Regression**
  - Used for **non-linear relationships**.
  - Fits a curved line (e.g., quadratic, cubic).
4. **Regularized Regression**
  - Used to prevent **overfitting**.
  - Applies **penalties** to reduce model complexity:
    - **Ridge**: Shrinks coefficients but keeps all.
    - **Lasso**: Shrinks and **eliminates** less relevant features.
    - **ElasticNet**: Combines both Ridge + Lasso.



## 5. Advanced Regression Algorithms

- **Random Forest Regression:** Ensemble of decision trees.
- **Support Vector Regression (SVR):** Finds a margin of tolerance for predictions.
- **Gradient Boosting:** Sequential tree-based improvements.
- **Neural Networks:** Uses layers of artificial neurons to predict complex outcomes.

---

### Examples

| Scenario                       | Regression Type           | Reason                     |
|--------------------------------|---------------------------|----------------------------|
| Predicting house prices 🏠      | Linear/Polynomial         | Continuous outcome         |
| Estimating temperature ☀️      | Regression                | Continuous & time-based    |
| Predicting final exam scores 📄 | Multiple regression       | Multiple features involved |
| Overfit model with noisy data? | Regularized (Lasso/Ridge) | To constrain coefficients  |

---

### Key Takeaways

- **Regression = continuous prediction**, while **classification = discrete class prediction**.
- In **classification**, classes are unordered (e.g., spam vs. not spam). In **regression**, outputs are ordered (e.g., 50 is greater than 30).
- **Evaluation Metrics:**
  - **Classification:** Accuracy, precision, recall, etc.
  - **Regression:** Error metrics like MAE, MSE, RMSE.
- Choose **regression** when your output is a real number (like price, weight, score).

---

## Lesson 6: Slope and Intercept

---

### What is This Lesson About?

This lesson teaches us how to **predict things using math**, like guessing someone's salary based on how many years they've worked. We use something called **linear regression**—but don't worry, it's just a fancy name for drawing a straight line through some dots to help us guess better!

---

## **Imagine This...**

Let's say you want to know how much money people make, based on how many years they've been working.

Here's what we know from the data:

### **Years of Experience    Salary (in \$1000)**

|         |          |
|---------|----------|
| 1 year  | \$45,000 |
| 2 years | \$50,000 |
| 3 years | \$55,000 |
| 4 years | \$60,000 |
| 5 years | \$65,000 |

Looks like more experience = more money! 🇺🇸

---

## **The Magic Equation: $y = mx + b$**

This is the superhero of this lesson!

- $y$  is the salary (what we're trying to guess)
  - $x$  is the number of years of experience
  - $m$  is the **slope** (how fast salary grows)
  - $b$  is the **intercept** (starting salary with 0 years)
- 

## **What's a Slope ( $m$ )?**

Think of slope like this:

Every year someone works, how much more money do they make?

Let's check from the table:

- From 1 year (\$45k) to 2 years (\$50k), they got \$5k more.

So the slope = **\$5,000 per year**

➡ This means **every year** of experience adds **\$5,000** to the salary.

---

## **What's an Intercept ( $b$ )?**

This is like the **starting point**.

If someone has **0 years of experience**, how much would they make?

We use the equation and plug in one point, like:

$$1 \text{ year} = \$45\text{k} \rightarrow$$

$$45 = 5(1) + b \rightarrow$$

$$b = 40$$

So the intercept is **\$40,000**.

Even with **no experience**, someone might start at **\$40k**.

---

### **Final Model (The Magic Formula)**

Now we put it all together:

$$\text{🧠 } y = 5x + 40$$

That means:

- Start at \$40k
- Add \$5k for every year of experience

Want to guess someone's salary? Just plug in the years!

📌 Example:

$$3 \text{ years} = 5(3) + 40 = 55 \rightarrow \textbf{\$55,000}$$

Boom! You're predicting like a pro 🤖

---

### **Imagine the Chart**

- Blue Dots: Real data (years and salaries)
- Red Line: The model (what we *predict* with our formula)

The red line goes through or near the blue dots—that's a good sign that our model is working well!

---

### **Conclusion (The Big Idea)**

- ✓ **Slope** = how fast salary grows
- ✓ **Intercept** = starting salary
- ✓ You can use math (linear regression) to **predict the future!**

Now you've learned how machines can "guess" real-world stuff using a simple line and a little math magic! 📊🔮

---

## 📖 Lesson 7: Classification

### 🧠 Objectives

- Define the **key concepts** of classification in machine learning.
- Understand the **types** of classification: binary vs. multiclass.
- Recognize the **difference** between classification and regression.
- List **common classification algorithms** and their characteristics.
- Know how to **evaluate** classification models.

---

### 🔗 Key Concepts / Steps

1. **Classification vs. Regression**
  - **Classification:** Predicts *categories* (e.g., *pass* or *fail*).  
→ Answers: "Which class does this belong to?"
  - **Regression:** Predicts *continuous values* (e.g., *exam score*).  
→ Answers: "What value should I expect?"
2. **What is Classification?**
  - A supervised ML task where you predict a **class label** based on **input features**.
  - Example: Predicting if a student will pass or fail based on attendance and study hours.
3. **Types of Classification**
  - **Binary Classification:** Two possible classes (e.g., spam vs. not spam).
  - **Multiclass Classification:** More than two classes (e.g., classifying fruit types or digits 1–9).
4. **Key Terms**
  - **Classifier:** The algorithm used to predict class labels.
  - **Feature:** An input variable used for prediction (e.g., study hours).
  - **Evaluation:** Assessing the performance of a model.
5. **Lazy vs. Eager Learners**
  - **Lazy Learner:**
    - Waits until test time to learn.
    - **Example:** K-Nearest Neighbors (KNN)
    - Slower prediction, no generalization.
  - **Eager Learner:**
    - Trains in advance, learns a general model.

- Faster prediction.
    - Examples: Logistic Regression, Decision Trees
  - 6. **Popular Classification Algorithms**
    - **K-Nearest Neighbors (KNN):**
      - Classifies by the majority vote of the  $k$  closest neighbors.
    - **Logistic Regression:**
      - Estimates the *probability* of a class.
    - **Decision Trees:**
      - Uses “if-then” logic to split data into class decisions.
    - **Other advanced models:**
      - Support Vector Machines (SVM)
      - Naive Bayes
      - Discriminant Analysis
      - Neural Networks
- 

## Examples

- **Binary classification:**

Predicting whether a student will *pass or fail* based on:

    - % of classes attended
    - Number of study hours
    - Average test scores
  - **KNN in action:**

A student’s data is compared to 5 other students. If 4 out of 5 similar students passed, the model predicts the new student will also *pass*.
  - **Logistic regression:**

Predicts the **probability** of passing based on features like attendance.
- 

## Key Takeaways

- **Classification** is for predicting **categories**, **regression** is for predicting **continuous numbers**.
  - Two main classification types: **binary** and **multiclass**.
  - ML algorithms used for classification include:
    - KNN (lazy learner)
    - Logistic Regression (probability-based)
    - Decision Trees (rule-based)
    - And more advanced models like SVM and neural networks.
  - Know your **features** (inputs), your **classes** (outputs), and your **evaluation** metrics to judge performance.
-

## 📘 Lesson 8: Evaluating Machine Learning Models

---

### 🧠 Objectives

By the end of this lesson, you will be able to:

- Define the **train/test split**.
  - Evaluate classification models using:
    - **Accuracy**
    - **Confusion Matrix**
    - **Precision**
    - **Recall**
    - **F1 Score**
  - Evaluate regression models using:
    - **Mean Squared Error (MSE)**
    - **Root Mean Squared Error (RMSE)**
    - **Mean Absolute Error (MAE)**
    - **R-squared (R<sup>2</sup>)**
- 

### ⚡ Key Concepts / Steps

#### 🔗 1. Train/Test Split

- Don't feed your model the *entire* dataset — it's not a buffet!
  - **Training Set**: Used to teach the model.
  - **Test Set**: Used to evaluate the model's performance on unseen data.
- 

#### 📊 2. Classification Evaluation Metrics

##### ✅ Accuracy

- Formula:

$$\text{Accuracy} = \frac{\text{Correct Predictions}}{\text{Total Predictions}}$$

- Example: 70% accuracy means the model got 7 out of 10 right.

##### 📋 Confusion Matrix

- A 2x2 matrix that compares **actual vs. predicted** labels.

|             | Predicted Pass      | Predicted Fail      |
|-------------|---------------------|---------------------|
| Actual Pass | True Positive (TP)  | False Negative (FN) |
| Actual Fail | False Positive (FP) | True Negative (TN)  |

- Helps you see where the model **excels** and where it **fails miserably** (gracefully, of course).

### Precision

- Measures the accuracy of **positive predictions**.
- Formula:

$$\text{Precision} = \frac{TP}{TP + FP}$$

- Example: If the model predicted 5 students to pass and only 4 really did, precision = 80%.

### Recall

- Measures how well the model finds **all actual positives**.
- Formula:

$$\text{Recall} = \frac{TP}{TP + FN}$$

- Example: If 6 students actually passed but the model got only 4, recall = ~66.7%.

### F1 Score

- The *harmonic mean* of precision and recall.
- Formula:

$$F1 = \frac{2 \cdot (\text{Precision} \cdot \text{Recall})}{\text{Precision} + \text{Recall}}$$

- Ideal when you need balance and can't afford to play favorites.

## 3. Regression Evaluation Metrics

### Mean Squared Error (MSE)

- Formula:

$$MSE = \frac{1}{n} \sum (y_{\text{actual}} - y_{\text{predicted}})^2$$

- Measures the **average squared difference** between actual and predicted values.
- Lower MSE = better model.

### Root Mean Squared Error (RMSE)

- Formula:

$$RMSE = \sqrt{MSE}$$

- Brings the error back to the **original unit** of measurement.

### Mean Absolute Error (MAE)

- Formula:

$$MAE = \frac{1}{n} \sum |y_{\text{actual}} - y_{\text{predicted}}|$$

- Easier to interpret, doesn't punish large errors as harshly as MSE.

### R-squared (R<sup>2</sup>)

- Formula:

$$R^2 = 1 - \frac{SS_{\text{res}}}{SS_{\text{tot}}} = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$$

- Measures the **proportion of variance** in the dependent variable explained by the independent variables.
- Value between 0 and 1:
  - 1 = perfect fit
  - 0 = no predictive power
- Aka "**Goodness of Fit**". (Not to be confused with your jeans size.)

### Examples

- **Classification (Pass/Fail):**
  - Accuracy: 70%
  - Precision: 4/5 = 80%
  - Recall: 4/6 ≈ 66.7%
  - Use **F1 Score** when both are important.
- **Regression (Exam Grades):**
  - MSE, RMSE, MAE give insight into error size.
  - R<sup>2</sup> tells you how *well* your model fits the data.



## ✿ Key Takeaways

- Always split your data into **training** and **testing** sets.
  - Accuracy is useful, but **confusion matrices** and **precision/recall** tell the full story.
  - Use **F1-score** when balancing precision and recall is critical.
  - For regression tasks:
    - **Lower errors** = better predictions.
    - **Higher  $R^2$**  = better fit.
- 

## 📖 Lesson 9: Clustering

### 🧠 Objectives

- Define **clustering** and its role in machine learning.
  - List common **applications** of clustering across industries.
  - Identify **types of clustering algorithms** and how they work.
  - Explain the **K-means** clustering algorithm step-by-step.
- 

### ⚡ Key Concepts / Steps

#### 🔍 What is Clustering?

- Clustering = Grouping **unlabeled** data based on **similarity** or **distance**.
  - It's like organizing your closet by color, type, or "things I might wear someday but probably won't."
  - **Unsupervised Learning** technique: No labels, no teacher — the algorithm learns from the structure.
- 

#### 🧠 Why Clustering?

To *discover hidden patterns, simplify datasets, or reveal relationships* that aren't obvious at first glance.

---

### 🌐 Real-World Applications

#### 1. 🏠 Customer Segmentation

- Group customers based on demographics & buying behavior.
- Businesses can tailor marketing like your Spotify playlists — *we know what you want before you do.*

2. 🖼️ **Image Segmentation**
    - Breaks images into meaningful parts by color, texture, etc.
    - Helpful in computer vision (think self-driving cars recognizing “tree” vs “pedestrian” 🚶 🌳).
  3. 🕵️ **Anomaly Detection**
    - Detects weirdos — outlier data points that don’t belong.
    - Great for fraud detection and cybersecurity.
  4. 📄 **Document Clustering**
    - Groups similar texts together (e.g., all political news in one folder, sports in another).
    - Used in search engines & content management.
  5. 💡 **Recommendation Systems**
    - Clusters similar users/products to suggest items.
    - Ever wonder how Netflix *knows* you’re in the mood for rom-coms? Yup — clustering.
- 

## 🧪 Types of Clustering Algorithms

1. **Partitioning Clustering**
    - Splits data into  $k$  predefined clusters.
    - You set  $k$ , the number of clusters — it’s like telling the algorithm how many boxes you want.
    - Examples:
      - **K-means**
      - **K-medoids**
  2. **Hierarchical Clustering**
    - Builds a tree-like structure (dendrogram).
    - Two flavors:
      - **Agglomerative** (bottom-up: start with each point as a cluster, merge as you go).
      - **Divisive** (top-down: start with one cluster, split it down).
    - No need to predefine  $k$ ! 🎉
  3. **Density-Based Clustering**
    - Clusters = areas of high point density.
    - Perfect for complex shapes and noise.
    - Examples:
      - **DBSCAN**
      - **OPTICS**
- 

## 📊 K-Means Algorithm: Step-by-Step

A fan favorite for a reason: fast, simple, and surprisingly effective — like the espresso shot of clustering ☕

1. **Initialize:** Randomly choose  $k$  data points as initial centroids.

2. **Assign:** Assign each data point to the nearest centroid (based on **Euclidean distance**).
3. **Update:** Recalculate each centroid as the mean of all points assigned to it.
4. **Repeat:** Reassign points and update centroids until the assignments stop changing (convergence).

🌀 Eventually, you'll get **k** stable clusters — one for each centroid.

---

### Examples (Conceptual)

- 📊 Grouping customers by age & spending habits for targeted promotions.
  - 🏠 Identifying regions in a medical image.
  - 📄 Categorizing online reviews by topics.
  - 🎧 Clustering songs based on mood or tempo.
- 

### Key Takeaways

- Clustering is an **unsupervised ML technique** for finding patterns in unlabeled data.
  - It's used in areas like **marketing, vision, security, content, and recommendation engines**.
  - There are multiple clustering approaches:
    - **Partitioning** (e.g., K-means, K-medoids)
    - **Hierarchical** (e.g., Agglomerative, Divisive)
    - **Density-Based** (e.g., DBSCAN, OPTICS)
  - **K-means** is a powerful, intuitive algorithm that divides data into  $k$  clusters based on distance from centroids.
- 

## Lesson 10: Generative AI: Overview and Use Cases

### Learning Objectives

- Define Generative AI (GenAI) and explain its significance.
  - Identify diverse real-world use cases across various industries.
- 

### What is Generative AI?

Generative AI (GenAI) is a branch of artificial intelligence that creates **entirely new content or data** — such as text, images, music, or even virtual worlds — using **deep learning techniques** and **large datasets**.

✦ **Key difference from traditional AI:**

While traditional AI operates based on rules and patterns, GenAI generates **original outputs** by learning from vast data.

---

✦ **Large Language Models (LLMs)**

Generative AI often incorporates **Large Language Models**, like ChatGPT, trained to understand and generate human language. These models:

- Support advanced natural language processing.
  - Can be part of larger systems to handle decision-making and creative tasks.
- 

🧠 **Benefits of Generative AI**

GenAI offers a range of strategic advantages:

- 🌟 **Creativity & Innovation**
  - 💰 **Cost and Time Efficiency**
  - 🧬 **Personalization**
  - 📈 **Scalability**
  - 🛡️ **Robustness**
  - 🔍 **Exploration of New Possibilities**
- 

🔧 **Use Cases Across Industries**

| Industry                   | Use Cases                                                                   |
|----------------------------|-----------------------------------------------------------------------------|
| Healthcare                 | Tailored treatments, genetic analysis, simulated surgeries, drug prediction |
| Agriculture                | Optimizing crop yields, creating resilient plant varieties                  |
| Biotechnology              | Drug discovery, forecasting drug efficacy                                   |
| Forensics                  | Analyzing DNA, identifying suspects                                         |
| Environmental Conservation | Analyzing endangered species' data, conservation planning                   |
| Creative Arts              | Generating digital art, music, videos for media & ads                       |
| Gaming                     | Dynamic level creation, character generation                                |
| Fashion                    | Virtual try-ons, personalized fashion recommendations                       |
| Robotics                   | Designing adaptive robot movements                                          |
| Education                  | Custom learning materials, adaptive learning environments                   |

| Industry     | Use Cases                                         |
|--------------|---------------------------------------------------|
| Data Science | Data augmentation to improve ML model performance |

## Summary

- **GenAI** enables machines to produce novel, high-quality content by learning from data.
- It provides **flexibility** and **adaptability** akin to human intelligence.
- Its impact spans **healthcare, art, education, robotics**, and more.
- GenAI represents a leap forward in how machines can assist and amplify human potential.

---

## Lesson 11: Generative AI Applications

### Objectives

- List **popular applications** of Generative AI.
- Understand **how each application** is used across different domains.

---

## Key Concepts / Steps

### ◆ What is Generative AI?

- It enables software to **create, generate, and simulate** new content using **machine learning** and **deep learning**.
- Unlike traditional rule-based systems, GenAI **learns patterns** from large datasets and produces **original, personalized content**.

---

## Key Applications of Generative AI

|  Application |  Description              |
|-------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------|
| <b>GPT (Generative Pre-trained Transformers)</b>                                                | Family of LLMs from OpenAI (e.g., GPT-3.5, GPT-4). Used in chatbots, writing tools, coding, journalism, etc. |
| <b>ChatGPT</b>                                                                                  | A conversational chatbot built on GPT. Assists with questions, writing, idea generation, and more.           |
| <b>Google Bard</b>                                                                              | A writing assistant using <b>LaMDA</b> , optimized for emails, content creation, and conversational tone.    |

|  Application |  Description                                              |
|-----------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| <b>IBM Watsonx</b>                                                                            | A suite for building and managing enterprise-grade AI: includes tools for training (Watsonx.ai), analyzing (Watsonx.data), and governance. |
| <b>DeepDream</b>                                                                              | Produces <b>psychedelic, surreal images</b> from real pictures — often used in digital art.                                                |
| <b>StyleGAN</b>                                                                               | Generates <b>high-quality synthetic faces</b> — used in gaming, virtual avatars, and medical training.                                     |
| <b>AlphaFold</b>                                                                              | Developed by DeepMind, predicts <b>3D protein structures</b> — revolutionizing biotech and drug development.                               |
| <b>Magenta (Google)</b>                                                                       | Creates AI-generated <b>music and art</b> , including AI-human music collaborations.                                                       |
| <b>Google PaLM 2</b>                                                                          | A next-gen LLM trained on massive datasets — excels in reasoning, coding, and language understanding.                                      |
| <b>GitHub Copilot</b>                                                                         | An AI coding assistant that <b>auto-completes and suggests code</b> in real time. Powered by OpenAI.                                       |

---

## Examples

-  **AlphaFold** has helped scientists predict protein structures at scale — vital for diseases like cancer and Alzheimer's.
  -  **StyleGAN** generated fake faces that look shockingly real — useful in design and research.
  -  **Magenta** composed original melodies and even performed duets with human musicians!
- 

## Key Takeaways

- **Generative AI empowers software** to generate unique, original content.
  - Its applications span **writing, coding, biology, art, gaming**, and more.
  - Major tools include **GPT, ChatGPT, Bard, AlphaFold, GitHub Copilot, and PaLM 2**.
  - While powerful, GenAI also raises ethical concerns around **misuse, deepfakes, and copyright**.
-

## Module One glossary:

### 1. Core Concepts in AI & Machine Learning

- **AI (Artificial Intelligence):** The field aiming to create machines that mimic human cognitive functions.
  - **Machine Learning:** A subfield of AI enabling computers to learn from data without explicit programming.
  - **Deep Learning:** A branch of ML where computers learn and make decisions autonomously using neural networks.
  - **Natural Language Processing (NLP):** Enables machines to understand and process human language.
  - **Generative AI:** Uses ML to generate original content by learning patterns from data.
  - **Augmented Intelligence:** AI designed to enhance human capabilities, not replace them.
  - **Large Language Model (LLM):** A deep learning model specialized in generating and understanding human language.
- 

### Types of Learning

- **Supervised Learning:** Models are trained on labeled data with known input-output pairs.
  - **Unsupervised Learning:** Models work with unlabeled data to identify hidden patterns or structures.
  - **Clustering:** An unsupervised learning technique that groups similar items based on features.
  - **Classification:** A supervised learning task that categorizes inputs into predefined classes.
  - **Regression:** A supervised technique for predicting continuous numerical values.
- 

### Algorithms & Techniques

- **Decision Tree:** A tree-like model used for classification and regression.
- **Random Forest:** An ensemble method combining multiple decision trees.
- **Support Vector Regression (SVR):** Uses support vectors to fit a regression line.
- **Gradient Boosting:** Builds models sequentially to correct previous errors.
- **k-Nearest Neighbors (KNN):** Classifies based on the majority class of nearby data points.
- **K-means Algorithm:** Divides data into K distinct clusters.
- **Density Estimation:** Estimates the probability distribution of a dataset.
- **Dimensionality Reduction:** Reduces the number of features while preserving important information.

---

## Model Building & Lifecycle

- **Feature Engineering:** Creating or transforming input variables to improve model performance.
  - **Feature Extraction:** Deriving meaningful features from raw data.
  - **Train/Test Split:** Dividing data into training and test sets to evaluate performance.
  - **Model Deployment:** Making the model available for real-world use.
  - **Machine Learning Model Lifecycle:** The end-to-end process of building, deploying, and maintaining models.
  - **Extract, Transform, Load (ETL):** Data preparation process for ML models.
- 

## Evaluation Metrics

- **Confusion Matrix:** Summarizes classification results with TP, TN, FP, FN counts.
  - **Precision:** Proportion of true positives among all predicted positives.
  - **Recall:** Proportion of true positives among all actual positives.
  - **F1-Score:** Harmonic mean of precision and recall.
  - **Mean Absolute Error (MAE):** Average of absolute prediction errors.
  - **Root Mean Squared Error (RMSE):** Square root of average squared errors.
  - **R-squared ( $R^2$ ):** Explains how well the independent variables explain the dependent variable.
  - **Squared Error:** Average of squared differences between predicted and actual values.
- 

## Mathematical & Statistical Terms

- **Dependent Variable:** The variable being predicted.
  - **Independent Variable:** The variable used for prediction.
  - **Line of Best Fit:** A line that best represents data in a scatter plot.
  - **Slope:** Rate of change between dependent and independent variables.
  - **Euclidean Distance:** A metric to measure distance between data points.
  - **Cluster Centroid:** The mean or median position representing a cluster.
- 

## Learning Styles

- **Eager Learner:** Trains thoroughly before prediction (e.g., decision trees).
- **Lazy Learner:** Defers learning until prediction time (e.g., KNN).
- **Classifier:** An algorithm that categorizes data into classes.



---

## Applications of ML

- **Anomaly Detection:** Identifying outliers or unusual patterns.
- **Image Segmentation:** Dividing images into segments based on features.
- **Recommendation Systems:** Suggesting items based on user behavior (often uses clustering).
- **Market Basket Analysis:** Finding relationships between items in transactions.

---

## Ethical & Responsible AI

- **Ethical Concerns:** Issues around misuse, fairness, IP, and privacy in AI applications.

---

## Visualizations & Tools

- **Scatter Plot:** A two-dimensional plot showing the relationship between variables.

---

## Lesson's Summary:

- Machine learning teaches computers to perform tasks without explicit programming.
- Machine learning includes supervised and unsupervised learning, with branches like deep learning, natural language processing, computer vision, and reinforcement learning.
- Data engineering involves collecting, storing, processing, and managing data for efficient access and analysis.
- The machine learning model lifecycle consists of problem definition, data collection, preparation, model development, evaluation, and deployment.
- Supervised learning uses labeled data for training, with techniques like classification and regression.

- Unsupervised learning uses unlabeled data to discover patterns and structures independently, with clustering as an example.
  - Classification and regression are prediction problems in machine learning.
  - Clustering groups unlabeled data to identify patterns or connections.
  - Clustering has applications in customer segmentation, image segmentation, anomaly detection, document clustering, and recommendation systems.
  - It's important to divide data into training and test sets.
  - A confusion matrix measures classification performance, while mean squared error evaluates regression models.
  - R-squared value represents the explained variance in a dependent variable by an independent variable.
  - Generative AI performs diverse tasks, rivaling traditional models in creativity, cost savings, personalization, scalability, robustness, and exploration.
- 

## Module Two

### Machine Learning with Apache Spark

#### Machine Learning with Apache Spark

#### Lesson 1: Spark for Data Engineers

#### Objectives

- Define Apache Spark and its core features.
  - Identify how Spark is used by data engineers.
  - Understand Spark's role in large-scale data processing and pipeline development.
- 

#### ✂ Key Concepts / Steps

##### 1. What is Apache Spark?

- A **distributed computing system** designed to process large-scale datasets efficiently.

- Developed to overcome the **slowness and limitations** of Hadoop MapReduce.
- Supports a variety of tasks: **batch processing, streaming, machine learning, and graph processing**.

## 2. Key Features of Spark:

-  **In-Memory Processing:** Stores data in memory for faster computation, reducing I/O costs.
-  **Multi-language Support:** APIs available in **Python, Java, Scala, and R**.
-  **Integration with Popular Data Sources:** HDFS, Amazon S3, Cassandra, and more.
-  **Scalability & Efficiency:** Handles petabyte-scale data with high performance.

## 3. Spark Tasks for Data Engineers:

-  **Data Ingestion:**
    - Intake data from HDFS, cloud storage, databases, or streaming platforms like **Apache Kafka**.
  -  **Data Transformation:**
    - Clean, filter, join, pivot, and aggregate data using Spark's powerful APIs (e.g., Spark SQL, DataFrame API).
  -  **Data Storage:**
    - Design distributed storage strategies using HDFS, S3, or databases like Cassandra.
  -  **Advanced Processing:**
    - Implement **machine learning, graph processing, and real-time streaming**.
  -  **Optimization:**
    - Tune partitioning, memory usage, caching, and optimize code for performance.
  -  **Monitoring:**
    - Monitor and troubleshoot Spark clusters using built-in and third-party tools.
- 

## Examples

No specific code examples in this lesson, but real-world applications include:

- Streaming customer behavior data from Kafka and transforming it in real-time.
  - Running ML models on Spark using distributed computing for scalable training.
-

## ✿ Key Takeaways

- **Spark** is a versatile distributed engine optimized for performance and scalability.
  - It empowers data engineers to **ingest, process, transform, store, and monitor** massive datasets.
  - Thanks to **in-memory computing** and multi-language support, Spark has become a central tool in modern data engineering workflows.
- 

## 📘 Lesson 2: Regression using SparkML

### 🧠 Objectives

- Understand what regression is in the context of Spark ML.
  - Learn how to build, train, and evaluate regression models using Spark's MLlib.
  - Identify the step-by-step process to apply regression in Spark.
- 

### ✂ Key Concepts / Steps

1. **Import Required Libraries**  
Import components like `SparkSession`, `VectorAssembler`, `LinearRegression`, and `RegressionEvaluator`.
2. **Create a SparkSession**  
Acts as the main entry point for using Spark functionality.
3. **Load the Dataset**  
Use `.read.csv()` with options like `header=True` and `inferSchema=True`.
4. **Select Relevant Columns**  
Choose feature columns (independent variables) and the target column (dependent variable).
5. **Create a VectorAssembler**  
Combine multiple feature columns into a single vector column (commonly named "features").
6. **Transform the Data**  
Apply the assembler to generate the "features" column for each row.
7. **Split the Data**  
Use `.randomSplit([0.7, 0.3], seed)` to divide data into training and testing sets.
8. **Create a Linear Regression Model Instance**  
Specify `featuresCol` and `labelCol` (target column name).
9. **Fit the Model**  
Train it on the training set using `.fit()`.
10. **Make Predictions**  
Use `.transform()` on the test set to generate predicted values.

### 11. Evaluate the Model

Use `RegressionEvaluator` to calculate metrics like **Root Mean Squared Error (RMSE)**.

### 12. Print Metrics & Model Parameters

- Print RMSE.
- Print model coefficients and intercept.

### 13. Stop SparkSession

Call `.stop()` to release resources.

---

## Example (Code Snippet Overview)

```
from pyspark.sql import SparkSession
from pyspark.ml.feature import VectorAssembler
from pyspark.ml.regression import LinearRegression
from pyspark.ml.evaluation import RegressionEvaluator

spark =
SparkSession.builder.appName("LinearRegressionExample").getOrCreate()

data = spark.read.csv("path_to_data.csv", header=True,
inferSchema=True)
data = data.select("feature1", "feature2", "target")

assembler = VectorAssembler(inputCols=["feature1", "feature2"],
outputCol="features")
assembled_data = assembler.transform(data)

train_data, test_data = assembled_data.randomSplit([0.7, 0.3],
seed=42)

lr = LinearRegression(featuresCol="features", labelCol="target")
model = lr.fit(train_data)

predictions = model.transform(test_data)

evaluator = RegressionEvaluator(labelCol="target",
predictionCol="prediction", metricName="rmse")
rmse = evaluator.evaluate(predictions)

print("Root Mean Squared Error:", rmse)
print("Coefficients:", model.coefficients)
print("Intercept:", model.intercept)

spark.stop()
```

---

## Key Takeaways

- Spark ML enables scalable, distributed regression modeling with ease.
- The process is modular: import → session → load → prepare → model → evaluate.
- RMSE is a key metric to judge model accuracy: **lower is better**.
- `VectorAssembler` is essential to handle multiple features.

- The Spark ML workflow is repeatable and flexible across different regression algorithms.
- 

## ■ Lesson 3: Classification using SparkML

### Objectives

- Define classification in the context of Spark ML.
  - Understand the steps to build, train, and evaluate a classification model using PySpark.
  - Learn how to use logistic regression for classification tasks in Spark.
- 

## ✂ Key Concepts / Steps

### 1. Import Required Libraries

- `SparkSession` from `pyspark.sql`
- `VectorAssembler` from `pyspark.ml.feature`
- `LogisticRegression` from `pyspark.ml.classification`
- `MulticlassClassificationEvaluator` from `pyspark.ml.evaluation`

### 2. Create `SparkSession`

This is your gateway to using all Spark functionalities, including DataFrames and ML.

### 3. Load Dataset

- Use `.read.csv()` method.
- Set `header=True` to use column names.
- Set `inferSchema=True` to detect datatypes automatically.

### 4. Select Feature and Target Columns

- Define a list of input columns (e.g., `feature_cols = ["age", "income"]`).
- Define the target column (e.g., `"label"`).

### 5. `VectorAssembler`

- Combine multiple input columns into a single `"features"` column.
- Syntax:
- `assembler = VectorAssembler(inputCols=feature_cols, outputCol="features")`
- `assembled_data = assembler.transform(data)`

### 6. Split Data

- Use `.randomSplit([0.8, 0.2], seed=42)` to create `train_data` and `test_data`.

### 7. Create Logistic Regression Model

- 8. `lr = LogisticRegression(labelCol="label", featuresCol="features")`

### 9. Train the Model

- 10. `model = lr.fit(train_data)`

### 11. Make Predictions

```
12. predictions = model.transform(test_data)
```

### 13. Evaluate Model

- o Use MulticlassClassificationEvaluator
- o Specify labelCol="label" and metricName="accuracy"

### 14. Print Accuracy

```
15. print("Accuracy:", evaluator.evaluate(predictions))
```

### 16. Stop SparkSession

```
17. spark.stop()
```

---



## Example (Code Snippet)

```
from pyspark.sql import SparkSession
from pyspark.ml.feature import VectorAssembler
from pyspark.ml.classification import LogisticRegression
from pyspark.ml.evaluation import MulticlassClassificationEvaluator

# Step 1 & 2: Create Spark Session
spark =
SparkSession.builder.appName("ClassificationExample").getOrCreate()

# Step 3: Load Dataset
data = spark.read.csv("path_to_dataset.csv", header=True,
inferSchema=True)

# Step 4 & 5: Select columns and assemble features
feature_cols = ["feature1", "feature2", "feature3"]
assembler = VectorAssembler(inputCols=feature_cols,
outputCol="features")
data = assembler.transform(data)

# Step 6: Split the data
train_data, test_data = data.randomSplit([0.8, 0.2], seed=42)

# Step 7: Create Model
lr = LogisticRegression(labelCol="label", featuresCol="features")

# Step 8: Train Model
model = lr.fit(train_data)

# Step 9: Make Predictions
predictions = model.transform(test_data)

# Step 10 & 11: Evaluate
evaluator = MulticlassClassificationEvaluator(labelCol="label",
metricName="accuracy")
accuracy = evaluator.evaluate(predictions)
print("Accuracy:", accuracy)

# Step 12: Stop Spark
spark.stop()
```

---

## ✿ Key Takeaways

- **Classification** predicts **categorical outcomes**, unlike regression which predicts continuous values.
  - Spark ML supports large-scale classification with distributed computing power.
  - Logistic Regression is a common starting point for classification tasks in PySpark.
  - `VectorAssembler` is essential for building the "features" vector.
  - Model evaluation uses `MulticlassClassificationEvaluator` with **accuracy** as a key metric.
  - A full pipeline includes: **data loading** → **preparation** → **model training** → **prediction** → **evaluation**.
- 

## 📘 Lesson 4: Clustering using SparkML

### 🧠 Objectives

- Define clustering using Spark ML.
  - Identify and execute the steps required to perform clustering using Spark ML.
- 

### ✂ Key Concepts / Steps

#### Clustering Overview

- **Definition:** Clustering is an *unsupervised learning* technique that groups similar data points based on intrinsic characteristics or patterns—no predefined labels are required.
- **Purpose:** Discover hidden structures within data.
- **Common Algorithms in Spark ML:**
  - **K-means** (most popular).
  - **Bisecting K-means**.
  - **Gaussian Mixture Models**.
- **Applications:** Customer segmentation, anomaly detection, image segmentation, recommendation systems.

#### Steps for Building a K-means Clustering Model in PySpark

1. **Import Libraries:**
  - `SparkSession` from `pyspark.sql`
  - `VectorAssembler` from `pyspark.ml.feature`
  - `KMeans` from `pyspark.ml.clustering`
2. **Create SparkSession** – entry point to Spark functionalities.



3. **Load Data into DataFrame** – typically from CSV; specify `header=True` and `inferSchema=True`.
  4. **Select Features** – define a list of columns to be used for clustering.
  5. **Assemble Feature Vectors** – use `VectorAssembler` to combine features into a single vector column (`features`).
  6. **Initialize K-means Model** – specify number of clusters `k` and random seed for reproducibility.
  7. **Train Model** – call `.fit()` on your dataset.
  8. **Make Predictions** – call `.transform()` to assign clusters to each data point.
  9. **Display Cluster Assignments** – select and show `features + prediction` columns.
  10. **Evaluate Clustering** – compute **silhouette score** to measure clustering quality.
  11. **Stop SparkSession** – free up resources.
- 

### Example

```
from pyspark.sql import SparkSession
from pyspark.ml.feature import VectorAssembler
from pyspark.ml.clustering import KMeans

# 1. Create Spark Session
spark =
SparkSession.builder.appName("ClusteringExample").getOrCreate()

# 2. Load Data
data = spark.read.csv("path_to_your_data.csv", header=True,
inferSchema=True)

# 3. Assemble Features
feature_cols = ["feature1", "feature2"]
assembler = VectorAssembler(inputCols=feature_cols,
outputCol="features")
data = assembler.transform(data)

# 4. Train K-means Model
kmeans = KMeans(k=3, seed=1, featuresCol="features",
predictionCol="prediction")
model = kmeans.fit(data)

# 5. Make Predictions
predictions = model.transform(data)

# 6. Show Cluster Assignments
predictions.select("features", "prediction").show()

# 7. Stop Session
spark.stop()
```

---

## ✿ Key Takeaways

- Clustering in Spark ML helps uncover patterns in unlabeled datasets by grouping similar data points.
  - K-means is the most commonly used algorithm, but Spark ML offers alternatives.
  - The process involves data preparation, feature assembly, model training, prediction, and evaluation.
  - The **silhouette score** is a standard metric to assess clustering quality.
  - Spark ML leverages distributed computing to handle clustering on large-scale datasets efficiently.
- 

## 📘 Lesson 5: Graph Frames on Apache Spark

### 🧠 Objectives

- Define **graph theory** and **GraphFrames**.
  - Identify data types suitable for GraphFrames.
  - Understand how GraphFrames works with Spark DataFrames.
  - Recognize common graph algorithms supported by GraphFrames.
- 

### ✂ Key Concepts / Steps

- **Graph Theory:** Mathematical study of relationships between objects, modeled as **vertices** (nodes) and **edges** (connections).
  - **Directed graphs:** One-way relationships (e.g., airline routes, manufacturing workflows).
  - **Undirected graphs:** Two-way relationships (e.g., social networks, genomics).
- **GraphFrames:**
  - Extension to Apache Spark for **graph processing** using **DataFrames**.
  - Differs from **GraphX**, which uses RDDs.
  - Supports Spark SQL APIs to query **vertex** and **edge** DataFrames.
- **Installation:**
  - Not included by default in Spark.
  - Download via package system (e.g., `--packages graphframes:graphframes:0.8.1-spark3.0-s_2.12`).
  - Ensure correct version for your Spark/Scala setup.
- **Features:**
  - Motif finding (`find` method) for detecting repeating patterns in a graph.
  - Built-in algorithms:
    - Breadth-First Search (BFS)
    - Connected Components

- Label Propagation Algorithm (LPA)
    - PageRank
    - Shortest Paths
    - Triangle Count
  - **Data suitability:** Any dataset that can be modeled as entities (**vertices**) connected by relationships (**edges**) with optional properties.
- 

## Examples

- **Social network:** Vertices = people, Edges = "friend" or "follows" relationship.
    - Example: Alice ↔ Esther (friends).
    - GraphFrames produces:
      - **Vertices DataFrame** → List of people.
      - **Edges DataFrame** → Relationships + properties.
    - Can run PageRank to determine influence or BFS to find connections.
- 

## Key Takeaways

- **Graph Frames** brings graph theory analysis into Spark via **DataFrames** and **Spark SQL**.
  - It's ideal for connected data scenarios like social networks, routes, or recommendation systems.
  - Includes **popular graph algorithms** for ranking, connectivity, and pattern detection.
  - Requires **separate installation** and proper version matching with Spark/Scala.
- 

## Module 2 Glossary

### 1. Clustering

- **Bisecting K-means** – Hierarchical clustering splitting clusters into smaller subclusters until desired number is reached.
  - **K-means** – Popular clustering algorithm assigning points to the nearest cluster mean.
  - **Gaussian Mixture Models (GMM)** – Probabilistic clustering using mixtures of Gaussian distributions.
  - **Silhouette score** – Measures clustering quality and separation between clusters.
-

## 2. Regression

- **Linear regression** – Models relationship between variables as a linear equation.
  - **Decision tree regression** – Uses a decision tree to model continuous target variables.
  - **Gradient-boosted tree regression** – Sequentially built ensemble of decision trees for better predictions.
  - **Random forest regression** – Combines multiple decision trees to improve regression accuracy.
  - **RegressionEvaluator** – Measures performance of regression models.
  - **Root Mean Squared Error (RMSE)** – Average prediction error magnitude.
- 

## 3. Classification

- **Logistic regression** – Models categorical outcomes using the logistic function.
  - **MulticlassClassificationEvaluator** – Evaluates performance of multiclass classification models.
  - **Sentiment analysis** – Classifies text as positive, negative, or neutral.
  - **Supervised learning** – Learns from labeled data to predict or classify unseen data.
- 

## 4. Spark Concepts & Architecture

- **Spark** – Distributed computing system for large-scale data processing.
  - **Spark Cluster** – Group of machines working together to process data in Spark.
  - **Spark Session / SparkSession** – Unified entry point for Spark features and APIs.
  - **Spark ML** – Spark's current machine learning library.
  - **Spark MLlib** – Older name for Spark ML.
  - **In-memory processing** – Stores data in memory to avoid slow disk I/O.
  - **Stream processing** – Handles real-time data streams.
  - **Hadoop MapReduce** – Older framework Spark replaces.
  - **Cluster monitoring** – Tracking Spark cluster performance and health.
- 

## 5. Data & Preprocessing

- **CSV file** – Comma-separated values text file format.
- **Data ingestion** – Importing large volumes of data into Spark.
- **DataFrame** – Distributed, column-oriented data structure in Spark.
- **DataFrameReader** – Loads data from various file formats into a DataFrame.

- **VectorAssembler** – Combines multiple feature columns into a single vector.
  - **Scaling** – Normalizing feature values to a consistent range.
- 

## 6. Graph Processing

- **GraphFrame** – Spark library for working with graph data.
  - **Domain-Specific Language (DSL)** – Special syntax for domain-specific tasks like GraphFrames motif finding.
  - **Vertex** – Node in a graph representing an entity.
- 

## 7. Tools

- **Jupyter Notebook** – Interactive environment for code, data, and documentation.
- 

## Lesson's Summary:

- Spark is a distributed computing system that handles large-scale data sets efficiently.
- Data engineers benefit from various features provided by Spark, which aid in creating scalable and effective data processing pipelines.
- With Spark's distributed computing approach, in-memory processing capabilities, and APIs, data engineers can tackle complex data structures and work with massive datasets.
- Classification using Spark ML is the task of predicting categorical labels or classes for a given set of features.
- Classification algorithms can be applied in various domains, such as sentiment analysis, spam detection, fraud detection, and image classification.
- Regression, using Spark ML, involves building and training regression models using the machine learning libraries offered by Spark.
- Regression algorithms enable the creation of models that predict continuous target variables using input features.
- Clustering, powered by Spark ML, involves grouping similar data points into clusters based on shared characteristics or patterns.

- Classification algorithms are well-suited for applications like customer segmentation, anomaly detection, image segmentation, and recommendation systems.
  - K-means algorithm instance specifies the number of clusters (k) and a random seed for reproducibility.
  - GraphFrames, integrated with Apache Spark, enables graph processing using DataFrames.
  - It provides separate DataFrames for graph vertices and edges, which can be analyzed with SparkSQL.
  - GraphFrames also includes popular built-in graph algorithms that can be applied to the edge and vertex DataFrames for various analysis tasks.
- 

## Module Three

### Data Engineering for Machine Learning with Apache Spark

### Data Engineering for Machine Learning with Apache Spark

#### Lesson 1: SparkSQL

#### Objectives

- Define **Spark SQL** and its purpose.
  - Identify the **steps to use Spark SQL** for DataFrame operations in PySpark.
- 

#### Key Concepts / Steps

1. **Import libraries** – `from pyspark.sql import SparkSession`.
  2. **Create a SparkSession** – Entry point for Spark SQL functionality.
  3. **Load data into a DataFrame** – Supports multiple formats (CSV, Hive, Parquet, JSON, JDBC).
  4. **Register DataFrame as a temporary view** – Enables SQL-like querying.
  5. **Execute SQL queries** – Using `spark.sql("SELECT ...")`.
  6. **Display results** – `.show()` to preview output in console.
  7. **Stop SparkSession** – Releases resources after execution.
-



## Examples

```
from pyspark.sql import SparkSession

# Step 1 & 2: Create SparkSession
spark = SparkSession.builder.appName("SparkSQLExample").getOrCreate()

# Step 3: Load CSV data into DataFrame
df = spark.read.csv("path_to_your_data.csv", header=True)

# Step 4: Register as temporary view
df.createOrReplaceTempView("my_table")

# Step 5: Run SQL query
result = spark.sql("SELECT column1, column2 FROM my_table")

# Step 6: Show results
result.show()

# Step 7: Stop SparkSession
spark.stop()
```

---



## Key Takeaways

- Spark SQL lets you run **SQL-like queries** directly on structured/semi-structured distributed data.
  - Integrates seamlessly with Spark's processing power and supports multiple **data sources**.
  - **Temporary views** allow treating DataFrames like SQL tables.
  - Optimized for **large-scale datasets**, improving performance of relational operations.
- 



## Lesson 2: ETL Workloads



## Objectives

- Define **ETL** (Extract, Transform, Load).
  - Explain how to perform ETL with Apache Spark.
-

## Key Concepts / Steps

### ETL Definition

- **Extract:** Obtain data from various sources (e.g., Parquet, Hive, JDBC, ORC, custom sources).
- **Transform:** Clean, filter, and reshape data into the required format (e.g., remove unneeded columns, trim whitespace, join tables).
- **Load:** Store transformed data into a target system (e.g., data warehouse, database).

### Spark ETL Workflow

1. **Extract**
  - Use Spark's built-in connectors for HDFS, Parquet, ORC, Hive, JDBC.
  - Can pull from multiple sources simultaneously.
  - Extendable API for custom data sources.
2. **Transform**
  - Use **Spark SQL** and DataFrame operations for filtering, grouping, aggregation, and formatting.
  - Prepare clean, analysis-ready datasets.
3. **Load**
  - Write results to storage (e.g., JDBC to Postgres, HDFS, Hive).
  - Same data source interfaces as in extraction.

---

## Examples

**Extract** (Parquet file into DataFrame):

```
df = spark.read.parquet("people.parquet")
```

**Transform** (Filter teenagers aged 13–19):

```
df.createOrReplaceTempView("people")
teenagers = spark.sql("SELECT * FROM people WHERE age BETWEEN 13 AND 19")
```

**Load** (Write to Postgres database):

```
teenagers.write \
    .format("jdbc") \
    .option("url", "jdbc:postgresql://localhost/dbname") \
    .option("dbtable", "teenagers") \
    .option("user", "username") \
    .option("password", "password") \
    .save()
```

---



## Key Takeaways

- ETL is fundamental for moving and shaping data between systems.
  - Apache Spark supports **native integration** with popular big data file formats and databases.
  - **Transformations** in Spark (via DataFrames or SQL) ensure clean, structured data before loading.
  - Spark scales efficiently from small to massive workloads, making it adaptable as data grows.
- 

## Lesson 3: Spark Structured Streaming

### Objectives

- Define **streaming data**.
  - Describe **Structured Streaming** in Spark.
  - Identify streaming **data sources** and **data sinks**.
  - Explain **output modes**.
  - Describe **streaming operations**, **listeners**, and **checkpointing**.
- 

## Key Concepts

### Streaming Data

- Continuously generated, often from multiple sources.
- Incomplete by nature; past data may be unavailable.
- Requires **incremental processing** as new data arrives.

### Apache Spark Structured Streaming

- Processes streams using **Spark SQL engine**.
  - Uses **DataFrame** and **Dataset APIs**.
  - Supports **micro-batch** and **continuous processing** modes.
  - Optimizes queries with the same SQL engine as batch jobs.
- 

## Core Terms

- **Source**: Origin of the data (e.g., file, Kafka, socket).
- **Sink**: Destination for output (e.g., Kafka, file, console).

- **Event Time:** Time a record was created (different from arrival or processing time).
  - **End-to-End Latency:** Time from data creation at the source to its output at the sink.
  - **Watermarking:** Handling **late data** by updating results after initial processing.
- 

## Data Sources

- Files
  - Kafka
  - IP Sockets
  - Rate sources
  - Custom sources
- 

## Output Modes

1. **Append** – Add only new rows.
  2. **Complete** – Rewrite the entire result table.
  3. **Update** – Update only changed rows (good for late data).
- 

## Data Sinks

- **Files** (directory output)
  - **Kafka** (topics)
  - **Foreach / ForeachBatch** (apply functions)
  - **Console / Memory** (*debugging only, not fault-tolerant*)
- 

## Operations

- SQL operations: `SELECT`, projections, aggregations.
  - Windowed aggregations over **event time**.
  - Joins: Stream–static and Stream–stream.
-

## Advanced Features

- **External Listeners:** Monitor streams and trigger actions.
  - **Checkpointing:** Store query state to recover from failures and resume processing.
- 

## Key Takeaways

- Structured Streaming uses **the same API** for batch and streaming.
  - Supports **multiple sources/sinks**, flexible **output modes**, and **fault tolerance**.
  - **Watermarking** is essential for handling late-arriving events.
- 

## Lesson 4: Feature Extraction and Transformation

### Objectives

- Define **data preprocessing**.
  - Describe **feature extraction** and steps in Spark.
  - Describe **feature transformation** and steps in Spark.
  - Understand **why** extraction & transformation are important.
- 

## Data Preprocessing

- Process of **cleaning, transforming, and preparing raw data** for analytics/ML.
  - Two key parts: **Feature Extraction & Feature Transformation**.
- 

## Feature Extraction

**Definition:** Selecting or deriving relevant features from raw data that capture important patterns or relationships, often correlated with the target variable.

### Common Techniques in Spark MLlib:

1. **Tokenization** – Split text into words/tokens. (`Tokenizer`)
2. **TF-IDF** – Score importance of words in a document. (`TF-IDF`)
3. **Word2Vec** – Represent words as numeric vectors. (`Word2Vec`)
4. **PCA** – Reduce dimensionality while preserving variance. (`PCA`)

### Steps in Spark:

1. Import Spark libraries (Spark SQL & Spark ML).
  2. Load data into **RDD** or **DataFrame**.
  3. Define extraction technique.
  4. Convert extracted features for analysis/ML tasks.
- 

## Feature Transformation

**Definition:** Convert extracted features into a more suitable form for modeling—improving quality, scaling, or representation.

### Common Techniques in Spark:

1. **Scaling & Normalization** – Bring numerical features to a common scale.
  - Tools: `StandardScaler`, `MinMaxScaler`, `MaxAbsScaler`
2. **One-Hot Encoding** – Convert categorical data to binary vectors. (`OneHotEncoder`)
3. **StringIndexer** – Assign numeric indices to categories. (`StringIndexer`)
4. **PCA** – Reduce dimensionality for computational efficiency and noise removal. (`PCA`)

### Steps in Spark:

1. Load data into **RDD** or **DataFrame**.
  2. Define transformation technique.
  3. Apply transformation.
  4. Format transformed data for analysis/ML tasks.
- 

## Importance

- Removes irrelevant/noisy information.
  - Improves **model accuracy**.
  - Enhances **training speed** and **inference performance**.
  - Helps with better **data representation** and **interpretability**.
-

## ■ Lesson 5: Data Engineering vs Machine Learning Pipelines

### 1. Data Engineering Pipelines

**Purpose:** Collect, clean, transform, integrate, and store raw data for analysis or ML.

**Key Components:**

1. **Data Collection** – From sources like databases, APIs, streaming, web scraping.
2. **Data Transformation** – Cleaning, handling missing values/outliers, normalization, standardization.
3. **Data Integration** – Merging datasets, joining related info, creating a unified view.
4. **Data Storage** – Using data warehouses or lakes for scalable, secure storage.

**Focus:** Data quality, scalability, smooth flow from raw sources to usable datasets.

---

### 2. Machine Learning Pipelines

**Purpose:** Train, evaluate, and deploy models using prepared data.

**Key Components:**

1. **Data Preprocessing** – Handling missing values, encoding categorical variables, scaling numeric data.
2. **Feature Engineering** – Selecting/creating features, applying dimensionality reduction.
3. **Model Training** – Applying algorithms to learn from data.
4. **Model Evaluation** – Using metrics (accuracy, MSE, precision, recall, F1) & cross-validation.
5. **Model Deployment** – Integrating models into production systems or APIs for real-time predictions.

**Focus:** Extracting insights and enabling predictive/automated decisions.

---

### 3. Relationship & Feedback Loop

- **Data Engineering → ML Pipelines:** Provides clean, structured data for training & inference.
  - **ML Pipelines → Data Engineering:** Feedback on data gaps or quality issues.
  - Continuous improvement loop between both pipelines.
-

## 4. Real-World Applications

- **Real-time Decision Making** – Fraud detection, recommendations, dynamic pricing.
  - **Scalable Data Processing** – Handling massive datasets for complex models.
  - **Model Deployment & Monitoring** – Ensuring data consistency, performance tracking, and integration.
- 

## 5. Core Insight

Data engineering builds the **data foundation**, and machine learning pipelines use that foundation to deliver **actionable intelligence**. Their synergy drives **innovation, efficiency, and competitiveness** in data-driven organizations.

---

### Lesson 6: Machine Learning Pipelines using Spark

#### **Definition:**

A systematic, automated workflow that organizes the entire ML process — from **data ingestion** to **model deployment**.

#### **Steps:**

1. **Data Ingestion** – Collect data from multiple sources (databases, files, APIs, streams).
2. **Data Preprocessing** – Clean data, handle missing values/outliers, scale & transform.
3. **Feature Extraction** – Select/create features relevant to the task.
4. **Model Selection & Training** – Choose and train the best ML algorithm.
5. **Model Evaluation** – Use metrics (accuracy, precision, recall, F1-score) & validation.
6. **Model Deployment** – Integrate into production, create APIs, monitor performance.

#### **Advantages:**

- **Reproducibility** – Documented, repeatable process.
  - **Scalability** – Handles large datasets efficiently.
  - **Automation** – Reduces manual repetitive work.
  - **Flexibility** – Swap algorithms/hyperparameters easily.
  - **Consistency** – Same preparation & modeling steps every run.
  - **Ease of Deployment** – Integrates well with production environments.
-

# Spark MLlib for Pipelines

## Key Benefits:

- Distributed computing for **scaling to large datasets**.
- **In-memory processing** for high performance.
- Unified interface for different data sources and ML algorithms.
- Rich library ecosystem.

## Pipeline Components in Spark:

- **Ingestion:** APIs for HDFS, S3, Cassandra, etc.
  - **Preprocessing:** Distributed transformations, missing value handling, filtering, scaling.
  - **Feature Extraction:** Tokenization, TF-IDF, word embeddings.
  - **Model Training:** Decision trees, random forests, logistic regression, etc.
  - **Evaluation:** Built-in metrics, cross-validation, performance reports.
  - **Deployment:** Export as JAR, PFA, or other portable formats.
- 

## Core Insight:

Machine learning pipelines bring **structure, automation, and scalability**, and Spark MLlib takes it further with **distributed, high-performance, big-data-ready tools** for every pipeline stage.

---

## Lesson 7: Model Persistence

**Model Persistence** is the process of **saving a trained machine learning model to disk** so it can be loaded later for predictions or further training.

## Advantages:

1. **Reusability** – Avoid retraining; use the same model across applications.
2. **Portability** – Share and load models across different systems or environments.
3. **Efficiency** – Load instantly without re-training, saving time and compute power.
4. **Reproducibility** – Consistently replicate results for verification and collaboration.
5. **Scalability** – Deploy at scale to handle large datasets or real-time predictions.
6. **Flexibility** – Use in diverse contexts (batch, streaming, web apps).

**Key takeaway:** Model persistence boosts productivity, reduces costs, and improves collaboration by making models reusable, shareable, and easy to integrate into production systems.

---

## ■ Lesson 8: Real-Time Use Case of Model Persistence

**Model Persistence** → Save a trained ML model to disk for future use → boosts **efficiency, scalability, flexibility**, enabling consistent results and collaboration.

**Use Case:** *Smart Home Energy Management System*

- Predicts energy consumption patterns and suggests optimal schedules.

**Benefits in Action:**

1. **Reusability** – Train once, reuse across homes without retraining.
2. **Portability** – Works on any smart hub or device brand.
3. **Efficiency** – Instant predictions and recommendations on installation.
4. **Reproducibility** – Same predictions across all homes; easy validation.
5. **Scalability** – Handles data from one home to whole neighborhoods via cloud.
6. **Flexibility** – Works for HVAC, lighting, appliances, adapting to different scenarios.

**Implementation Steps:**

- Train & save model → Load & reuse → Share → Deploy at scale → Integrate into multiple smart home systems.

**Outcome:** Optimized energy use, cost savings, reliable predictions, and a seamless smart home experience.

---

## ■ Glossary:

### 1. Spark Structured Streaming

- **Checkpointing** – Mechanism for recovering query progress in case of node failures and writing stream data to disk.
- **End-to-end latency** – Time required for data to process from source to sink.
- **Streaming data** – Continuously generated data from multiple sources requiring incremental processing.
- **Watermarking** – Process for managing late-arriving data and updating results after initial processing.

---

### 2. Spark SQL

- **Code snippet** – Small section of code demonstrating specific functionality.
- **Header** – First row of a file containing column names in tabular data.



- **JDBC (Java Database Connectivity)** – Java API for interacting with relational databases using SQL.
  - **Result set** – Output from executing an SQL query on a DataFrame.
  - **SQL queries** – Statements written in SQL syntax to retrieve, manipulate, and analyze data in a DataFrame.
- 

### 3. Feature Extraction and Transformation

- **Factor analysis** – Statistical method to identify latent factors underlying observed variables.
  - **IDF (Inverse Document Frequency)** – Measures rarity of a term across documents.
  - **Inference** – Applying a trained ML model to make predictions on unseen data.
  - **MaxAbsScaler** – Scales numerical features by their maximum absolute value.
  - **MinMaxScaler** – Scales numerical features to a specified range, typically 0–1.
  - **One-hot encoding** – Converts categorical features into numerical form.
  - **PCA (Principal Component Analysis)** – Reduces dataset dimensions while retaining variance.
  - **Scaling and normalization** – Transforms numerical features into a common scale to avoid bias.
  - **StandardScaler** – Scales numerical features to zero mean and unit variance.
  - **TF (Term Frequency)** – Frequency of a term in a document relative to total terms.
  - **TF-IDF** – Combines term frequency with inverse document frequency to measure term importance.
  - **Tokenization** – Breaks text into smaller units like words or phrases.
  - **Word2Vec** – Represents words as vectors capturing semantic relationships.
- 

### 4. Machine Learning Pipelines Using Spark

- **Distributed computing** – Distributes computations across multiple nodes for large-scale data processing.
  - **Machine learning pipeline** – Structured approach from data ingestion to model deployment.
  - **Model selection and training** – Choosing and training the right ML model using preprocessed data.
  - **Unified programming interface** – Spark’s consistent interface for working with varied data sources and algorithms.
-

## 5. Model Persistence

- **Model persistence** – Saving a trained ML model to disk for reuse and deployment.
  - **Portability** – Loading a saved model in different environments.
  - **Reproducibility** – Replicating ML experiments using the same saved model.
  - **Scalability** – Deploying saved models to handle large data volumes efficiently.
- 

## 6. Data Warehousing and ETL

- **Data warehouse** – Central repository storing data from multiple sources for analysis and reporting.
- 

### Lesson's Summary:

- Apache Spark Structured Streaming supports the processing of streaming data using Spark SQL.
- Feature extraction and transformation are crucial components of data preprocessing.
- Feature extraction involves selecting or deriving significant features from raw data for analysis or modeling purposes.
- MLlib provides various feature extraction techniques such as Tokenization, TF-IDF, Word2Vec, and PCA.
- Feature transformation involves converting extracted features into a suitable format for subsequent analysis.
- Spark offers commonly used feature transformation techniques like Scaling and normalization, One-hot encoding, StringIndexer, and PCA.
- Machine learning pipelines automate and structure the end-to-end ML process.
- They involve interconnected steps like data ingestion, cleaning, preprocessing, and so on.
- Advantages of ML models include reproducibility, scalability, automation, flexibility, and many more.

- Spark provides powerful tools for building and deploying ML models at scale.
  - Its key components simplify ML tasks, providing advantages such as scalability, performance, and ease of use.
  - Model persistence is an important concept in machine learning that involves saving a trained model to disk for future use.
  - Model persistence is an essential component of machine learning pipelines.
  - By enabling the saving and reuse of trained models, it enhances machine learning systems' efficiency, scalability, and flexibility.
  - Its advantages collectively contribute to improved productivity, reproducibility, and collaboration in the field of machine learning.
-